

Lecture 3: MLP와 오류역전파학습 (교재 Chapter 2 신경망)

<기계학습 개론> 2019 강의
서울대학교 컴퓨터공학부
장 병 탁

교재: 장교수의 딥러닝, 홍릉과학출판사, 2017.

Biointelligence Laboratory
School of Computer Science and Engineering
Seoul National University



목차

3.1 다층퍼셉트론 구조 (교재 Ch 2.3)	3
3.2 오류역전파 알고리즘 (교재 Ch 2.4)	9
3.3 활용 사례 (교재 Ch 2.5)	18
3.4 참고 1: 모멘텀, 가중치감소, 조기중단	24
3.5 참고 2: 지지벡터머신(SVM)	25
요약	29

3.1 다층퍼셉트론의 구조 (교재 Ch 2.3) (1/6)

■ 다층 뉴런의 필요성

- 복잡한 패턴 분류를 위해서는 입출력 간의 복잡한 변환 구조가 필요
- 단일 뉴런으로는 **선형분리**가 가능한 결정 경계선만을 생성 가능
- 사용하는 뉴런의 수를 늘리고 층을 추가하여
복잡한 구조의 의사결정 경계(decision boundary)를 생성할 수 있음

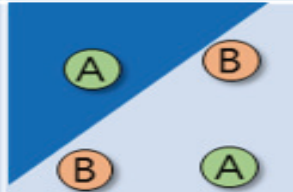
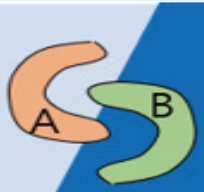
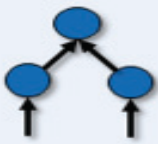
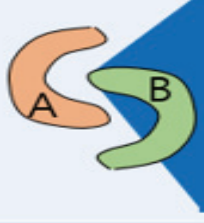
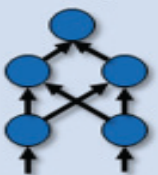
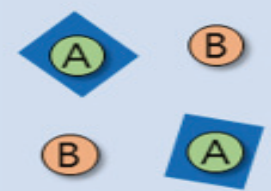
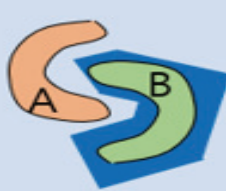
■ 다층퍼셉트론 (Multilayer Perceptron, MLP)

- 여러 개의 퍼셉트론 뉴런을 여러 층으로 쌓은 다층신경망 구조
- 각 층 안에서는 뉴런간 연결이 없으며 **인접한 두 층의 뉴런** 간에는 **완전 연결**됨
- 층별 활성화값
 - j : 시냅스후 뉴런, i : 시냅스전 뉴런

$$s_j = \sum_i w_{ji} x_i \quad f_j = \sigma(s_j) = \frac{1}{1 + \exp(-s_j)}$$

3.1 다층퍼셉트론의 구조 (2/6)

■ 다수 / 다층 뉴런의 필요성

Structure	Regions	XOR	Meshed Regions
Single layer 	Halfplane bounded by hyperplane		
Two layers 	Convex Open or closed regions		
Three layers 	Arbitrary (limited by # of nodes)		

Multiple boundaries needed
(e.g. XOR problem)

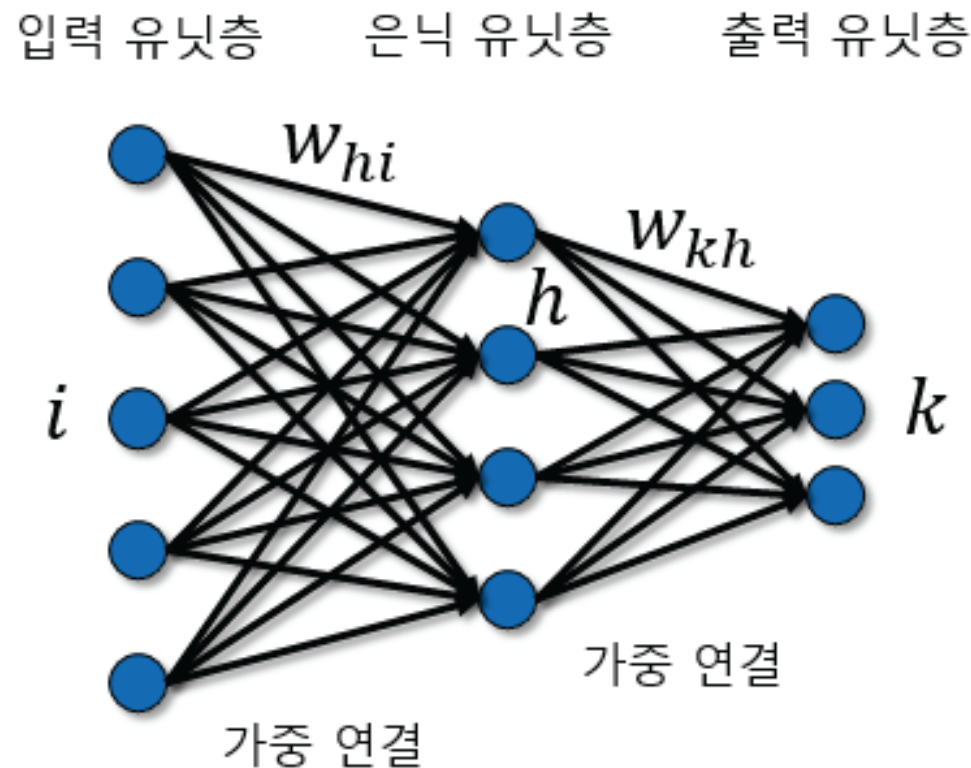
→ **Multiple units**

More complex regions needed
(e.g. Polygons)

→ **Multiple layers**

3.1 다층퍼셉트론의 구조 (3/6)

■ 다층퍼셉트론 신경망(MLP)



3.1 다층퍼셉트론의 구조 (4/6)

■ 다층퍼셉트론의 혁신

- 시그모이드 뉴런의 사용
 - 비선형 모델로 확장 가능
 - 연속함수이므로 미분 가능
 - 은닉 뉴런층의 학습을 가능하게 하는 오류역전파 알고리즘의 개발로 이어짐
- 은닉 뉴런층 추가
- 은닉층 학습방법 도입

3.1 다층퍼셉트론의 구조 (5/6)

■ 다층신경망 모델

- 입력층, 은닉층, 출력층으로 구성
 - 각 층의 뉴런 개수가 각각 I, H, K 라 하면 I - H - K 다층퍼셉트론이라 부름
- 입력유닛, 은닉유닛, 출력유닛을 가리키는 기호로 i, h, k 를 각각 사용
 - 입력 뉴런 i 의 출력은 모든 은닉 뉴런 h 와의 연결 가중치 를 통해서 전달됨
 - 은닉 뉴런 h 의 출력은 모든 출력 뉴런 k 의 입력으로 연결선 를 통해서 전달 됨
- 매개변수
 - w_{hi} : 입력층과 은닉층을 연결하는 가중치 벡터
 - w_{kh} : 은닉층과 출력층을 연결하는 가중치 벡터

3.1 다층퍼셉트론의 구조 (6/6)

■ 다층신경망 모델

■ 출력값 벡터 f

$$\mathbf{f} = (f_1, f_2, \dots, f_K) = \mathbf{f}(\mathbf{x}, \mathbf{w})$$

■ 출력값의 계산과정

■ 은닉 뉴런의 출력값 x_h 의 계산

$$s_h = \sum_{i=0}^I w_{hi}^{(1)} x_i \quad x_h = \sigma(s_h) = \frac{1}{1 + \exp(-s_h)}$$

■ 출력 뉴런의 값 f_k 의 계산

$$s_k = \sum_{h=0}^H w_{kh}^{(2)} x_h \quad f_k = \sigma(s_k) = \frac{1}{1 + \exp(-s_k)}$$

$$f_k = f_k(\mathbf{x}, \mathbf{w}) = \sigma \left(\sum_{h=0}^H w_{kh}^{(2)} \sigma \left(\sum_{i=0}^I w_{hi}^{(1)} x_i \right) \right)$$

3.2 오류역전파 알고리즘 (1/9)

■ 출력 뉴런 j 에 대한 가중치 벡터 변경

$$\mathbf{w}_j \leftarrow \mathbf{w}_j + \Delta \mathbf{w}_j$$

$$\Delta \mathbf{w}_j = \eta(y_j - f_j)f_j(1 - f_j)\mathbf{x}_j$$

■ $x_i^{(l-1)}$: $l - 1$ 층에 있는 뉴런들의 활성화값

■ $s_j^{(l)}$: l 층의 j 번째 뉴런의 총입력값

$$s_j^{(l)} = \mathbf{w}_j^{(l)} \cdot \mathbf{x}_j^{(l-1)} = \sum_i w_{ji}^{(l)} x_i^{(l-1)}$$

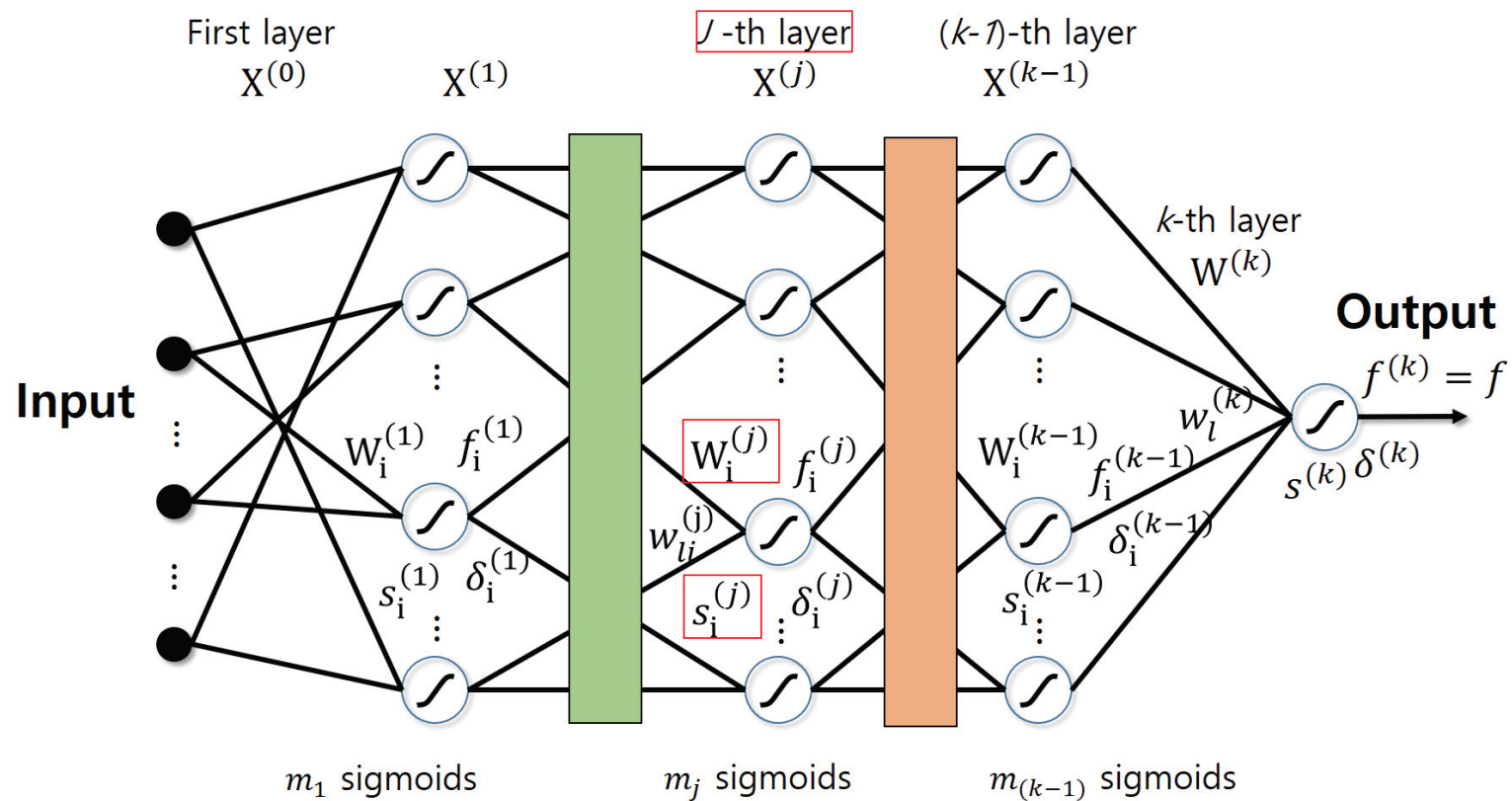
$$x_j^{(l)} = f_j^{(l)} = \sigma(s_j^{(l)})$$

$$\frac{\partial s_j^{(l)}}{\partial \mathbf{w}_j^{(l)}} = \mathbf{x}^{(l-1)}$$

$$\frac{\partial f_j^{(l)}}{\partial s_j^{(l)}} = f_j^{(l)}(1 - f_j^{(l)})$$

3.2 오류역전파 알고리즘 (2/9)

■ 단순 퍼셉트론에서 다층 퍼셉트론으로



3.2 오류역전파 알고리즘 (3/9)

■ 출력 뉴런 j 에 대한 가중치 벡터 변경(cont.)

- d 번째 학습예에 대한 오차함수

$$E_d = \frac{1}{2} \sum_j (y_j - f_j^{(\ell)})^2$$

- 출력층 d 층에 있는 j 번째 뉴런의 델타값 $\delta_j^{(\ell)}$
 - 해당 뉴런의 입력총합에 대한 에러변화율의 음수값
 - 교정할 오류를 의미

$$\delta_j^{(\ell)} \triangleq -\frac{\partial E_d}{\partial s_j^{(\ell)}}$$

3.2 오류역전파 알고리즘 (4/9)

■ 델타값의 계산

- 연쇄규칙을 사용
- 목표출력치가 주어지는 출력뉴런에 대한 델타값

$$\frac{\partial E_d}{\partial s_j^{(\ell)}} = \frac{\partial E_d}{\partial f_j^{(\ell)}} \frac{\partial f_j^{(\ell)}}{\partial s_j^{(\ell)}} = -(y_j - f_j^{(\ell)}) f_j^{(\ell)} (1 - f_j^{(\ell)})$$

$$\delta_j^{(\ell)} = (y_j - f_j^{(\ell)}) f_j^{(\ell)} (1 - f_j^{(\ell)})$$

- 출력층 l 층에 있는 j 번째 뉴런에 연결된 시냅스의 경사도

$$\begin{aligned} \frac{\partial E_d}{\partial \mathbf{w}_j^{(\ell)}} &= \frac{\partial E_d}{\partial f_j^{(\ell)}} \frac{\partial f_j^{(\ell)}}{\partial s_j^{(\ell)}} \frac{\partial s_j^{(\ell)}}{\partial \mathbf{w}_j^{(\ell)}} = -(y_j - f_j^{(\ell)}) \frac{\partial f_j^{(\ell)}}{\partial s_j^{(\ell)}} \mathbf{x}_j^{(\ell-1)} = -(y_j - f_j^{(\ell)}) f_j^{(\ell)} (1 - f_j^{(\ell)}) \mathbf{x}_j^{(\ell-1)} \\ &= -\delta_j^{(\ell)} \mathbf{x}_j^{(\ell-1)} \end{aligned}$$

3.2 오류역전파 알고리즘 (5/9)

■ 델타값의 계산(cont.)

- 은닉층에서는 목표출력치가 없으므로 델타값을 추정해야 함
- 은닉층 l 층에 있는 j 번째 뉴런의 가중치 수정

$$\frac{\partial E_d}{\partial \mathbf{w}_j^{(l)}} = \frac{\partial E_d}{\partial s_j^{(l)}} \frac{\partial s_j^{(l)}}{\partial \mathbf{w}_j^{(l)}} = -\delta_j^{(l)} \mathbf{x}_j^{(l-1)}$$

- $l + 1$ 층의 델타값 $\delta_k^{(l+1)}$ 을 이용한 $\delta_j^{(l)}$ 의 계산

$$\delta_j^{(l)} = -\frac{\partial E_d}{\partial s_j^{(l)}}$$

$$\begin{aligned} \frac{\partial E_d}{\partial s_j^{(l)}} &= \sum_{k \in \text{Upper}(j)} \frac{\partial E_d}{\partial s_k^{(l+1)}} \frac{\partial s_k^{(l+1)}}{\partial f_j^{(l)}} \frac{\partial f_j^{(l)}}{\partial s_j^{(l)}} = \sum_{k \in \text{Upper}(j)} -\delta_k^{(l+1)} w_{kj}^{(l+1)} f_j^{(l)} (1 - f_j^{(l)}) \\ &= -f_j^{(l)} (1 - f_j^{(l)}) \sum_{k \in \text{Upper}(j)} \delta_k^{(l+1)} w_{kj}^{(l+1)} \end{aligned}$$

3.2 오류역전파 알고리즘 (6/9)

■ 델타값의 계산(cont.)

- l 층의 j 번째 뉴런에 연결된 시냅스 가중치 벡터의 일반 변경식

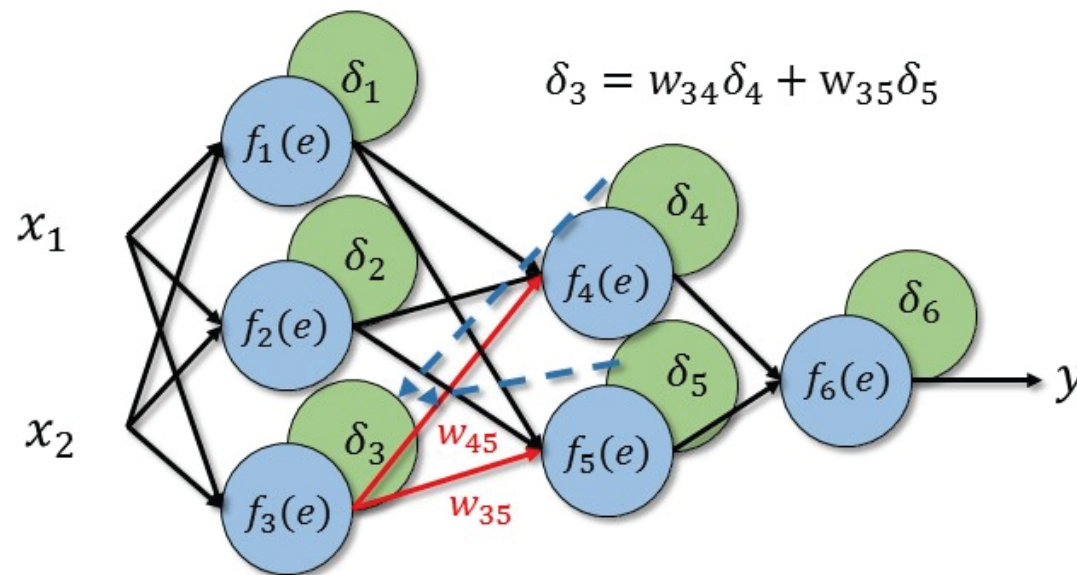
$$\mathbf{w}_j^{(\ell)} \leftarrow \mathbf{w}_j^{(\ell)} + \Delta \mathbf{w}_j^{(\ell)}$$

$$\Delta \mathbf{w}_j^{(\ell)} = -\eta \frac{\partial E_d}{\partial \mathbf{w}_j^{(\ell)}} = \eta \delta_j^{(\ell)} \mathbf{x}_j^{(\ell-1)}$$

$$\delta_j^{(\ell)} = \begin{cases} f_j^{(\ell)}(1-f_j^{(\ell)})(y_j - f_j^{(\ell)}) & \text{for } j \in \text{Outputs} \\ f_j^{(\ell)}(1-f_j^{(\ell)}) \sum_{k \in \text{Upper}(j)} \delta_k^{(\ell+1)} w_{kj}^{(\ell+1)} & \text{for } j \in \text{Hiddens} \end{cases}$$

3.2 오류역전파 알고리즘 (7/9)

- 상위층의 델타값을 이용한 하위층 뉴런의 델타값 계산



3.2 오류역전파 알고리즘 (8/9)

■ 오류역전파 알고리즘의 한계

- 학습속도가 느림
- 극복방법: **모멘텀** 항을 추가하여 학습을 가속(Yin, 1996)(Bishop, 2006)

$$\Delta w_{ji}(t) = \eta \delta_j x_{ji}$$

$$\Delta w_{ji}(t) = \eta \delta_j x_{ji} + \alpha \Delta w_{ji}(t-1), \quad 0 < \alpha < 1$$

3.2 오류역전파 알고리즘 (9/9)

오류역전파(error back-propagation) 학습 알고리즘

Given: 학습 데이터 $D = \{(\mathbf{x}^{(d)}, \mathbf{y}^{(d)})\}_{d=1}^N$, 학습률 η , 다층퍼셉트론 구조 $f(\mathbf{x}, \mathbf{w})$

Find: 다층퍼셉트론 연결가중치 벡터 $\mathbf{w}_{BP} = \arg \min_{\mathbf{w}} \left\{ \frac{1}{2} \sum_{d=1}^N \sum_{k=1}^K \left(f_k(\mathbf{x}^{(d)}, \mathbf{w}) - y_k^{(d)} \right)^2 \right\}$

Procedure: 다층퍼셉트론 학습

1. 연결가중치 w_{ji} 를 임의의 작은 값으로 초기화한다.
2. 입력 데이터 $\mathbf{x}^{(d)}$ 하나를 입력층에 넣고 활성화값을 전방으로 전달하면서 출력 값 $f_k(\mathbf{x}^{(d)}, \mathbf{w}), k = 1, \dots, K$ 를 계산한다.
3. 각 출력 뉴런 k 에 대한 오차 $\delta_k = (f_k - y_k)f_k(1 - f_k)$ 를 계산한다
($f_k = f_k(\mathbf{x}^{(d)}, \mathbf{w})$).
4. 출력 뉴런 k 에 대한 연결 가중치 값의 증분 $\Delta w_{kh} = -\eta \frac{\partial E_d}{\partial w_{kh}} = \eta \delta_k x_h$ 을 계산한다.
5. 은닉 뉴런 h 에 대한 오차 $\delta_h = f_h(1 - f_h) \sum_k w_{kh} \delta_k$ 를 계산한다.
6. 은닉 뉴런 h 에 대한 연결 가중치의 증분 $\Delta w_{hi} = -\eta \frac{\partial E_d}{\partial w_{hi}} = \eta \delta_h x_i$ 를 계산한다.
7. 가중치의 값을 모두 수정한다.
 $w_{ji} \leftarrow w_{ji} + \Delta w_{ji} \text{ for } j \in \text{Upper}(i)$

3.3 활용 사례 (1/6)

■ 다층신경망의 활용

- 다양한 감독 학습 문제에 적용 가능
 - 입력과 출력이 다차원인 경우
 - 입력과 출력이 이산값 / 연속값인 경우
- MNIST 데이터셋에 적용(LeCun, Cortes, & Burges, 1998)
 - 입력: 숫자 이미지 픽셀값의 벡터
 - 출력: 0부터 9까지의 숫자 클래스 값
- 스팸 메일 여과기 만들기(Clark, Koprinska, & Poon, 2003)
 - 입력: 메일에 나오는 단어들에 대한 빈도수 벡터
 - 출력: 스팸인지 아닌지를 판별하는 두 개의 클래스 값

3.3 활용 사례 (2/6)

■ 신경망의 응용 사례

Face recognition
Object recognition
Voice recognition
Spam-mail detection
Disease detection



3.3 활용 사례 (3/6)

■ 다층신경망의 구체적인 활용 사례: 얼굴 방향 인식

- 학습 데이터: 20명의 다른 사람을 촬영한 624장의 흑백 이미지 (Mitchell, 1997)
 - 가로 120픽셀, 세로 128픽셀의 이미지를 가로 30픽셀, 세로 32픽셀로 가공
 - 신경망의 입력층 뉴런 수를 줄여 계산 부담을 더는 효과
 - 각 픽셀은 0에서 255 사이의 밝기를 가짐
 - 사진 속 얼굴이 바라보는 방향은 상, 좌, 우, 정면 4가지로 분류

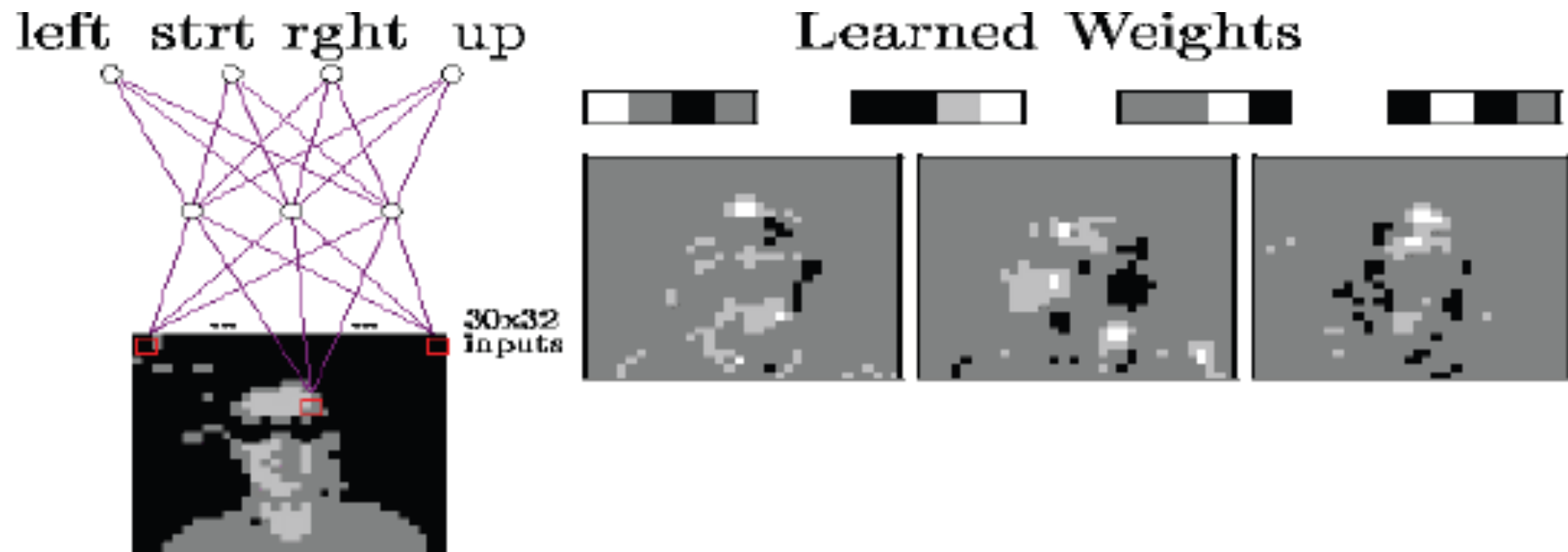


3.3 활용 사례 (4/6)

- 다층신경망의 구체적인 활용 사례: 얼굴 방향 인식(cont.)
 - 실험에 사용한 모델
 - 입력층 뉴런: 가공 이미지의 픽셀 수와 같은 960개
 - 은닉층 뉴런: 3개
 - 출력층 뉴런: 4개
 - 각 뉴런이 얼굴이 바라보는 방향 중 하나에 대응됨
 - 가장 큰 값을 보이는 뉴런의 방향이 이미지 속 얼굴 방향과 같도록 학습을 진행
 - 학습 방법
 - 신경망의 가중치는 크기가 작은 랜덤 값으로 초기화
 - 학습 가속을 위해 경사도 계산시 모멘텀 항 추가
 - 학습을 50번 반복할 때마다 검토용 데이터(validation set)에 대한 성능을 측정, 가장 높은 인식 정확도를 보이는 신경망을 얻음

3.3 활용 사례 (5/6)

- 얼굴 방향 인식 문제를 학습한 신경망과 학습된 가중치의 분포



3.3 활용 사례 (6/6)

- 다층신경망의 구체적인 활용 사례: 얼굴 방향 인식(cont.)
 - 은닉층의 경우 뉴런 개수를 자유롭게 조절하여 모델 성능에 변화를 줄 수 있음
 - Mitchell의 실험 결과
 - 은닉층 뉴런이 3개인 경우
 - 학습 시간: 5분
 - 테스트 데이터에 대한 방향 인식 정확도: 90%
 - 은닉층 뉴런이 30개인 경우
 - 학습 시간: 60분
 - 테스트 데이터에 대한 방향 인식 정확도: 91.5%

3.4 참고 1: 모멘텀, 가중치감소, 조기중단

■ BP의 문제점과 해결책

- 수렴속도가 느림
 - Momentum Term
 - Hessian
- 과다학습
 - Weight Decay
 - Stochastic Descent
 - Early Stopping
- 국지적 최소점만 찾음
 - Global Search
 - Simulated Annealing
 - Genetic Algorithm

$$\Delta w_{ji}(n) = \eta \delta_j x_{ji}$$

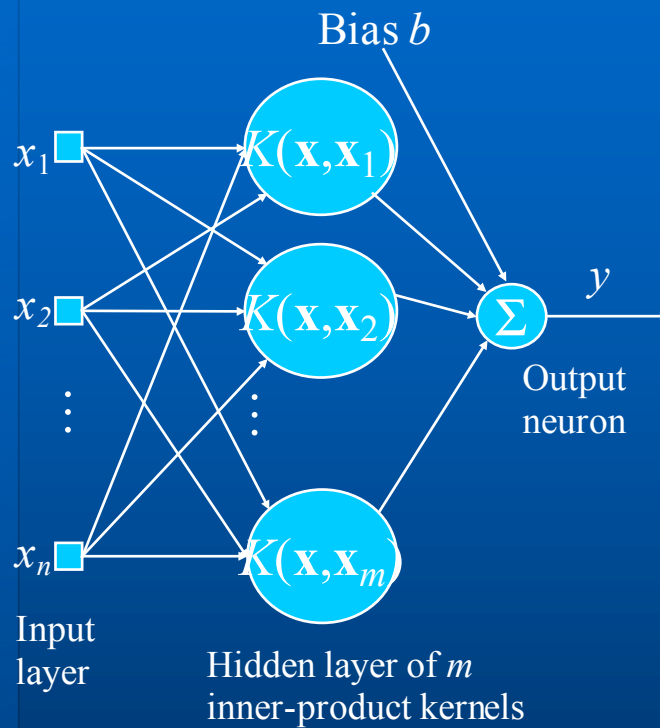
$$\Delta w_{ji}(n) = \eta \delta_j x_{ji} + \alpha \Delta w_{ji}(n-1), \quad 0 < \alpha < 1$$

$\Delta E(\mathbf{w}(n)) \approx \mathbf{g}^T(n) \Delta \mathbf{w}(n) + \frac{1}{2} \Delta \mathbf{w}^T(n) \mathbf{H}(n) \Delta \mathbf{w}(n)$
 $\mathbf{g}(n)$ is the m -by-1 gradient vector of the cost function $E(\mathbf{w})$ evaluated at the point $\mathbf{w}(n)$. The matrix $\mathbf{H}(n)$ is the m -by- m Hessian matrix of $E(\mathbf{w})$ (second derivative), $\mathbf{H} = \nabla^2 E(\mathbf{w})$

$$P(\gamma) = \frac{e^{-E_\gamma/T}}{Z(T)},$$

$$Z(T) = \sum_{\gamma'} e^{-E_{\gamma'}/T},$$

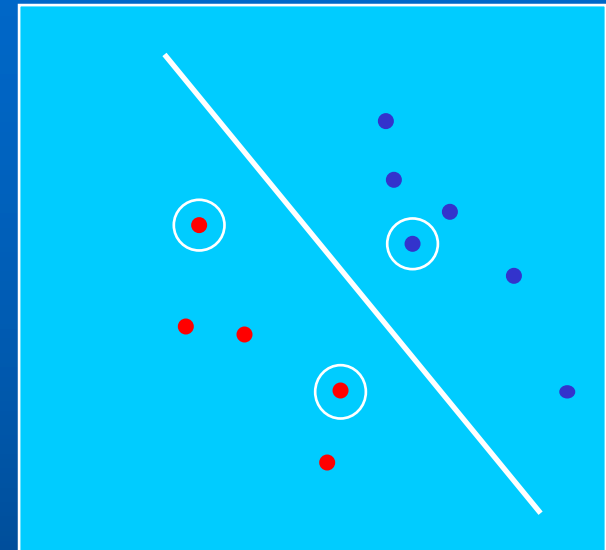
3.5 참고 2: 지지벡터머신(SVM)



Type of SVM	Inner product kernel $K(\mathbf{x}, \mathbf{x}_i)$
Polynomial learning machine	$(1 + \mathbf{x}^T \mathbf{y})^p$
Radial-basis function network	$\exp\left(-\frac{1}{2\sigma^2} \ \mathbf{x} - \mathbf{x}_i\ ^2\right)$
Two-layer perceptron	$\tanh(\beta_0 \mathbf{x}^T \mathbf{x}_i + \beta_1)$

Support Vector Machines

- The line that maximizes the minimum margin is a good bet.
 - ◆ The model class of “hyper-planes with a margin of m ” has a low VC dimension if m is big.
- This maximum-margin separator is determined by a subset of the datapoints.
 - ◆ Datapoints in this subset are called “support vectors”.
 - ◆ It will be useful computationally if only a small fraction of the datapoints are support vectors, because we use the support vectors to decide which side of the separator a test case is on.



The support vectors are indicated by the circles around them.

Training a linear SVM

- To find the maximum margin separator, we have to solve the following optimization problem:

$$\mathbf{w} \cdot \mathbf{x}^c + b > +1 \quad \text{for positive cases}$$

$$\mathbf{w} \cdot \mathbf{x}^c + b < -1 \quad \text{for negative cases}$$

$$\text{and } \|\mathbf{w}\|^2 \text{ is as small as possible}$$

- This is tricky but it's a convex problem. There is only one optimum and we can find it without fiddling with learning rates or weight decay or early stopping.
 - ◆ Don't worry about the optimization problem. It has been solved. Its called quadratic programming.
 - ◆ It takes time proportional to N^2 which is really bad for very big datasets
 - so for big datasets we end up doing approximate optimization!

Support Vector Machines

- SVM optimization

$$\begin{aligned} \min_{\mathbf{w}, b, \xi} \quad & 0.5 \|\mathbf{w}\|^2 + C \sum_{i=1}^m \xi_i \\ \text{subj to:} \quad & y_i (\langle \mathbf{w}, \phi(x_i) \rangle + b) \geq 1 - \xi_i \\ & \xi_i \geq 0 \end{aligned}$$

- Addresses generalization issue but not the computational cost of dealing with large vectors

요약 (Lecture 3)

- 다층퍼셉트론(MLP)은 단순 퍼셉트론에 은닉층을 추가한 신경망 구조이다.
- 다층퍼셉트론의 은닉층을 학습시킬수 있는 오류역전파 알고리즘(BP)이 1986년에 개발됨에 따라서 비선형의 복잡한 함수 근사 문제나 패턴 분류 문제를 풀수 있게 되었다.
- 오류역전파 알고리즘은 감독학습 알고리즘으로서 답을 아는 출력 뉴런에 대한 오차를 계산후 이를 역으로 입력층쪽으로 전파함으로써 순차적으로 (답을 모르는) 은닉층의 오차를 추정하여 이를 교정하는 방식이다.
- BP를 사용한 MLP는 다양한 패턴인식 문제에 성공적으로 적용되었다.
- BP의 단점은 수렴속도가 느리고 과다학습될 수 있다는 것이다. 이 문제를 해결하기 위해서 모멘텀항, 가중치 감소법, 조기중단법, 확률적 경사하강법 등 다양한 방법들이 연구되었다.
- 1990년대 중반에 지지벡터머신과 커널머신의 등장하여 BP의 속도 문제와 과다학습 문제점 등을 모두 해결하는 것처럼 보였다.
- 그러나 후에 (2012년경에) 딥러닝은 SVM이 해결하지 못하는 아주 복잡한 학습 문제들을 해결할 수 있음을 보였다.